

Лабораторна робота №4

Рекурентні нейронні мережі

Мета роботи: вивчити побудову та навчання рекурентної нейронної мережі на Python з використанням функцій бібліотеки `numpy`. Вивчити приклад побудови мережі для вирішення задачі аналізу настроїв. Самостійно вивчити програмні функції, які зустрічаються у прикладі RNN. Побудувати свою RNN відповідно до завдання (див. п 10. Завдання).

1. Введення. Теоретичні засади побудови RNN.

Спочатку розглянемо RNN, яка працює за принципом "*багато до багатьох*" (Рис.1). Вхідні дані – $x_0, x_1, \dots, x_n$, а результати виводу – $y_0, y_1, \dots, y_n$. Дані x_i та y_i є векторами і можуть бути довільних розмірів.

Рекурентні нейронні мережі RNN працюють шляхом ітерованого оновлення прихованого стану h , яке є вектором і може мати довільний розмір. Необхідно враховувати, що на будь-якому заданому часовому етапі t :

1. Поточний прихований стан h_t обчислюється на основі попереднього h_{t-1} та поточного входу x_t ;
2. Поточний вихід y_t обчислюється з урахуванням h_t .

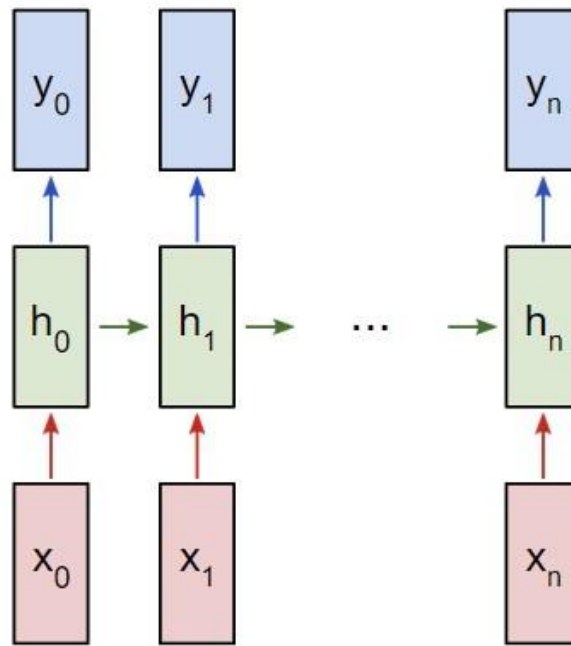


Рис. 1. Рекурентна нейронна мережа RNN "багато до багатьох"

Дана мережа на кожному кроці **використовує один й той самий набір вагових коефіцієнтів**. Точніше, типова класична рекурентна нейронна мережа використовує лише три набори параметрів ваги для виконання необхідних підрахунків:

- W_{xh} використовується для всіх зв'язків $x_t \rightarrow h_t$
- W_{hh} використовується для всіх зв'язків $h_{t-1} \rightarrow h_t$
- W_{hy} використовується для всіх зв'язків $h_t \rightarrow y_t$

Для рекурентної нейронної мережі ми також використовуємо два зміщення (зсува):

- b_h додається при підрахунку h_t
- b_y додається при підрахунку y_t

Ваги будуть представлені у вигляді матриці, а зміщення у вигляді вектору. Тобто, в даному випадку рекурентна нейронна мережа складається з трьох параметрів ваги та двох зсувів:

$$\begin{aligned} h_t &= \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \\ y_t &= W_{hy}h_t + b_y \end{aligned} \tag{1}$$

У цих формулах використовується матричне множення та застосовується функція гіперболічного тангенсу як функція активації (можливе також використання сигмоїдної функції).

2. Подготовка набора навчальних та тестових даних.

Скористаємося набором даних про емоційні стани, що передаються в тексті, який складається з двох словників (наборів даних), представлених за посиланням: <https://github.com/vzhou842/rnn-from-scratch/blob/master/data.py>

Фрагмент цього набору даних представлений Таблиці.1.

Таблица 1.

Текст	Позитивный?
Я хороший	Да
Я плохой	Нет
Это очень хорошо	Да
Это неплохо	Да
Я плохой, а не хороший	Нет
Я несчастен	Нет
Это было хорошо	Да
Я чувствую себя неплохо, мне не грустно	Да

3. Розробка архітектури RNN.

У мережі, що розробляється, буде використана архітектура "багато до одного". Принцип її роботи нагадує роботу архітектури "багато до багатьох", описаної вище. Однак цього разу буде задіяно лише один вихід у (рис.2):

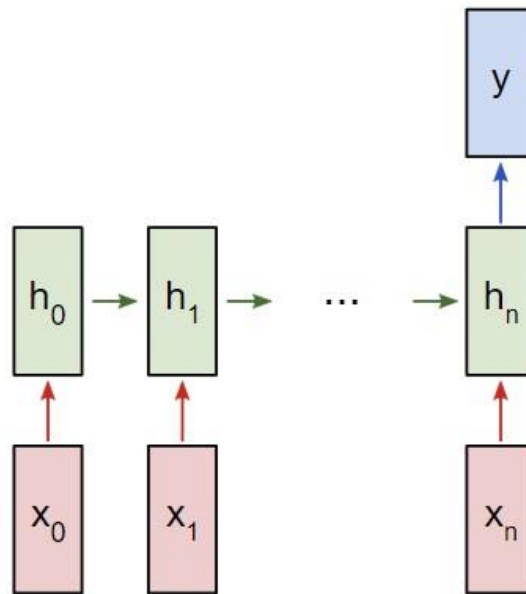


Рис.2. RNN "багато до одного"

Кожен x_i буде вектором, що представляє **певне слово** з тексту (у реченні), тоді n – кількість слів у реченні, що розглядається. Вихід y буде вектором, що містить два числа. Одне репрезентує позитивний настрій, а друге — негативне. Ми використовуємо функцію *Softmax*, щоб перетворити ці значення на ймовірність, і, зрештою, здійснимо класифікацію позитивного чи негативного контексту в текстовій фразі.

4. Створення словників з наборів даних.

Два набори даних: навчальний та тестовий (див. розділ 2) мають такий вигляд у Python:

```

1 train_data = {
2     'good': True,
3     'bad': False,
4     # ... больше данных
5 }
6
7 test_data = {
8     'this is happy': True,
9     'i am good': True,
10    # ... больше данных
11 }

```

True = Позитивное, False = Негативное

Для отримання даних у зручному форматі потрібно здійснити певну попередню обробку. Для початку необхідно створити словник (див. що таке [словарь в Python](#)) з усіх слів, які вживаються в наборах даних:

```

1 from data import train_data, test_data
2
3 # Создание словаря
4 vocab = list(set([w for text in train_data.keys() for w in text.split(' ')]))
5 vocab_size = len(vocab)
6
7 print('%d unique words found' % vocab_size) # найдено 18 уникальных слов

```

`vocab` тепер містить перелік всіх слів, які вживаються щонайменше в одному навчальному тексті. Далі надамо кожному слову з `vocab` індекс типу `integer` (ціле число).

```

1 # Назначить индекс каждому слову
2 word_to_idx = { w: i for i, w in enumerate(vocab) }
3 idx_to_word = { i: w for i, w in enumerate(vocab) }
4
5 print(word_to_idx['good']) # 16 (это может измениться)
6 print(idx_to_word[0]) # грустно (это может измениться)

```

Тепер можна відобразити будь-яке задане слово за допомогою індексу - цілого числа, оскільки рекурентна нейронна мережа не розрізняє слів – лише числа. Кожен вхід x_i для рекурентної нейронної мережі, що розглядається, є вектором. Будемо використовувати вектори, які представлені, як унітарний код (див. що таке [унітарний код](#)). Одиниця в кожному векторі x_i буде знаходитись у відповідному цілочисельному індексу слова. Оскільки у словнику 18 (див. табл.1) унікальних слів, кожен x_i буде 18-разрядним унітарним вектором. Повний текст програм, які розглядаються далі, можна переглянути за посиланням [GitHub](#).

```
1 import numpy as np
2
3
4 def createInputs(text):
5     '''
6     Возвращает массив унитарных векторов
7     которые представляют слова в введенной строке текста
8     - текст является строкой string
9     - унитарный вектор имеет форму (vocab_size, 1)
10    '''
11
12    inputs = []
13    for w in text.split(' '):
14        v = np.zeros((vocab_size, 1))
15        v[word_to_idx[w]] = 1
16        inputs.append(v)
17
18    return inputs
```

Ми також будемо використовувати `createInputs()` для створення вхідних даних у вигляді векторів та подальшої їх передачі в рекурентну нейронну мережу RNN.

5. Фаза прямого розповсюдження сигналів в нейронній мережі.

Створимо рекурентну нейронну мережу. Почнемо ініціалізацію з трьома параметрами ваги та двома зсувами.

```

1 import numpy as np
2 from numpy.random import randn
3
4
5 class RNN:
6     # Классическая рекуррентная нейронная сеть
7
8     def __init__(self, input_size, output_size, hidden_size=64):
9         # Вес
10        self.Whh = randn(hidden_size, hidden_size) / 1000
11        self.Wxh = randn(hidden_size, input_size) / 1000
12        self.Why = randn(output_size, hidden_size) / 1000
13
14        # Смещения
15        self.bh = np.zeros((hidden_size, 1))
16        self.by = np.zeros((output_size, 1))

```

Зверніть увагу: щоб нормалізувати або обмежити ваги (прибрати їх внутрішню варіативність), ми їх ділимо на 1000. Це не найкращий спосіб ініціалізації ваг, але він досить простий і непогано працює для даного прикладу.

Для ініціалізації ваг використовуємо стандартний нормальний розподіл, застосовуючи функцію [np.random.randn\(\)](#). Потім ми реалізуємо пряме поширення сигналів у нейронній мережі, що розглядається (див. формули (1)). Ці ж рівняння, реалізовані в коді:

```

1 class RNN:
2     # ...
3
4     def forward(self, inputs):
5         """
6         Выполнение передачи нейронной сети при помощи входных данных
7         Возвращение результатов вывода и скрытого состояния
8         Вывод - это массив одного унитарного вектора с формой (input_size, 1)
9         """
10        h = np.zeros((self.Whh.shape[0], 1))
11
12        # Выполнение каждого шага в нейронной сети RNN
13        for i, x in enumerate(inputs):
14            h = np.tanh(self.Wxh @ x + self.Whh @ h + self.bh)
15
16        # Compute the output
17        y = self.Why @ h + self.by
18
19        return y, h

```

Зверніть увагу на те, що ми ініціалізували h для нульового вектора на першому кроці (оскільки ми не маємо попереднього h), який тепер можна використовувати. Далі:

```
1 # ...
2
3 def softmax(xs):
4     # Применение функции Softmax для входного массива
5     return np.exp(xs) / sum(np.exp(xs))
6
7
8 # Инициализация нашей рекуррентной нейронной сети RNN
9 rnn = RNN(vocab_size, 2)
10
11
12 inputs = createInputs('i am very good')
13 out, h = rnn.forward(inputs)
14 probs = softmax(out)
15 print(probs) # [[0.50000095], [0.49999905]]
```

Наша рекурентна нейронна мережа працює, тепер перейдемо до навчання.

6. Фаза зворотного поширення помилки у нейронній мережі

Для навчання рекурентної нейронної мережі буде використано функцію втрат, а саме **втрату перехресної ентропії**, яка в більшості випадків сумісна з функцією *Softmax*.

Формула для розрахунку функції:

$$L = -\ln(p_c)$$

Тут p_c є ймовірністю передбачення рекурентною нейронною мережею класу correct (позитивний або негативний). Наприклад, якщо текст класифікується рекурентною нейронною мережею як позитивний на 90%, то втрата становитиме:

$$L = -\ln(0.90) = 0.105$$

За наявності параметрів втрати можна навчити нейронну мережу так, щоб вона використовувала градієнтний спуск для мінімізації втрат.

7. Параметри RNN.

Параметри даних, які будуть використані надалі:

- y — необроблені вихідні дані мережі;
- p — кінцева ймовірність: $p = \text{softmax}(y)$;
- c — істинна мітка визначеного зразка тексту («вірний» клас);
- L — втрата перехресної ентропії: $L = -\ln(p_c)$;
- W_{xh} , W_{hh} и W_{hy} — три матриці ваг у мережі;
- b_h и b_y — два вектори зсуву у мережі RNN.

Наступним кроком буде налаштування фази прямого розповсюдження. Це необхідно для кешування окремих даних, які будуть використовуватися у фазі зворотного розповсюдження помилки у нейронній мережі. Код виглядатиме так:

```

class RNN:
    # ...

    def forward(self, inputs):
        """
        Выполнение фазы прямого распространения нейронной сети с
        использованием введенных данных.
        Возврат итоговой выдачи и скрытого состояния.
        - Входные данные в массиве однозначного вектора с формой (input_size, 1)
        """
        h = np.zeros((self.Whh.shape[0], 1))

        self.last_inputs = inputs
        self.last_hs = { 0: h }

        # Выполнение каждого шага нейронной сети RNN
        for i, x in enumerate(inputs):
            h = np.tanh(self.Wxh @ x + self.Whh @ h + self.bh)
            self.last_hs[i + 1] = h

        # Подсчет вывода
        y = self.Why @ h + self.by

        return y, h

    def backprop(self, d_y, learn_rate=2e-2):
        """
        Выполнение фазы обратного распространения нейронной сети RNN.
        - d_y (dL/dy) имеет форму (output_size, 1).
        - learn_rate является вещественным числом float.
        """
        pass

```

8. Обчислення градієнтів

Почнемо з обчислення $\frac{\partial L}{\partial y}$

$$L = -\ln(p_c) = -\ln(\text{softmax}(y_c))$$

Тут використовується фактичне значення $\frac{\partial L}{\partial y}$, а також застосовується диференціювання функції. Отримуємо наступне:

$$\frac{\partial L}{\partial y_i} = \begin{cases} p_i & \text{if } i \neq c \\ p_i - 1 & \text{if } i = c \end{cases}$$

Наприклад, якщо $n = [0.2, 0.2, 0.6]$, а коректним класом є $c = 0$, то кінцевим результатом буде значення $\frac{\partial L}{\partial y} = [-0.8, 0.2, 0.6]$. Даний вираз може бути представлений у наступному коді:

```

1  # Цикл для кожного прикладу тренування
2  for x, y in train_data.items():
3      inputs = createInputs(x)
4      target = int(y)
5
6      # Прямое распространение
7      out, _ = rnn.forward(inputs)
8      probs = softmax(out)
9
10     # Создание dL/dy
11     d_L_d_y = probs
12     d_L_d_y[target] -= 1
13
14     # Обратное распространение
15     rnn.backprop(d_L_d_y)

```

Тепер розберемося з градієнтами для W_{hy} та b_y , які використовуються тільки для переходу кінцевого прихованого стану в результат виходу нейронної мережі RNN, що розглядається. Використовуємо такі вирази:

$$\frac{\partial L}{\partial W_{hy}} = \frac{\partial L}{\partial y} * \frac{\partial y}{\partial W_{hy}}$$

$$y = W_{hy}h_n + b_y$$

Тут h_n є кінцевим прихованим станом. Таким чином:

$$\frac{\partial y}{\partial W_{hy}} = h_n$$

$$\frac{\partial L}{\partial W_{hy}} = \boxed{\frac{\partial L}{\partial y} h_n}$$

Аналогічним чином обраховуємо:

$$\frac{\partial y}{\partial b_y} = 1$$

$$\frac{\partial L}{\partial b_y} = \boxed{\frac{\partial L}{\partial y}}$$

Тепер можемо приступати до реалізації `backprop()`:

```

1 class RNN:
2     # ...
3
4     def backprop(self, d_y, learn_rate=2e-2):
5         '''
6         Выполнение фазы обратного распространения нейронной сети RNN.
7         - d_y (dL/dy) имеет форму (output_size, 1).
8         - learn_rate является вещественным числом float.
9         '''
10        n = len(self.last_inputs)
11
12        # Подсчет dL/dWhy и dL/dby.
13        d_Why = d_y @ self.last_hs[n].T
14        d_by = d_y

```

Нагадування: ми створили `self.last_hs` в `forward()` в попередніх прикладах коду.

Нарешті, нам знадобляться градієнти для W_{hh} , W_{xh} , та b_h , які використовувались на кожному кроці нейронної мережі. У нас є формула:

$$\frac{\partial L}{\partial W_{xh}} = \frac{\partial L}{\partial y} \sum_t \frac{\partial y}{\partial h_t} * \frac{\partial h_t}{\partial W_{xh}}$$

Изменение влияет не только на каждый но и на все y , что, в свою очередь, приводит к изменениям в L . Для того, чтобы полностью подсчитать градиент необходимо провести *обратное распространение через все временные шаги*. Его также называют **Обратным распространением во времени**, или Backpropagation Through Time (BPTT) (Рис.3):

Зміна W_{xh} впливає не тільки на кожен h_t , а й на всі y , що, своєю чергою, призводить до змін у L . Щоб повністю підрахувати градієнт W_{xh} , необхідно провести *зворотне поширення* крізь всі часові кроки. Його також називають **Зворотним поширенням у часі**, або **Backpropagation Through Time (BPTT)** (Рис.3):

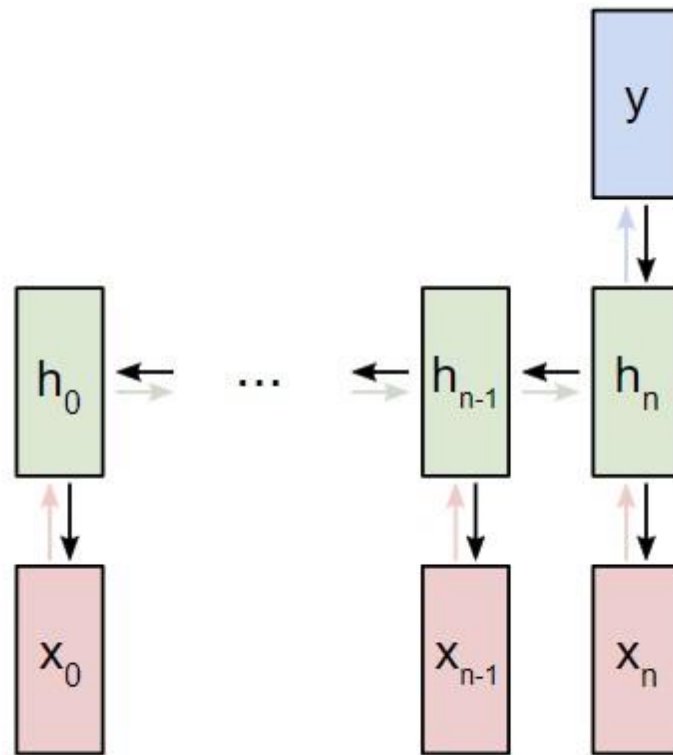


Рис.3. Схема зворотного поширення у часі.

W_{xh} використовується для всіх прямих посилянь $x_t \rightarrow h_t$, тому нам треба провести зворотнє розповсюдження назад до кожного з цих посилянь.

Наблизившись до заданого кроку t , треба обрахувати $\frac{\partial h_t}{\partial W_{xh}}$:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Похідна гіперболічної функції $\tanh$ нам вже відома:

$$\frac{d \tanh(x)}{dx} = 1 - \tanh^2(x)$$

Використовуємо диференціювання складної функції, або ланцюгове правило:

$$\frac{\partial h_t}{\partial W_{xh}} = (1 - h_t^2)x_t$$

Аналогічним чином обчислюємо:

$$\frac{\partial h_t}{\partial W_{hh}} = (1 - h_t^2)h_{t-1}$$

$$\frac{\partial h_t}{\partial b_h} = (1 - h_t^2)$$

Останнє потрібне значення — $\frac{\partial y}{\partial h_t}$. Його можна обчислити рекурсивно:

$$\begin{aligned} \frac{\partial y}{\partial h_t} &= \frac{\partial y}{\partial h_{t+1}} * \frac{\partial h_{t+1}}{\partial h_t} \\ &= \frac{\partial y}{\partial h_{t+1}} (1 - h_t^2) W_{hh} \end{aligned}$$

Реалізуємо зворотне поширення у часі, або ВРТТ, відштовхуючись від прихованого стану у якості початкової точки. Далі працюватимемо у зворотному порядку. Тому на момент підрахунку $\frac{\partial y}{\partial h_t}$ значення $\frac{\partial y}{\partial h_{t+1}}$ буде відоме. Винятком стане лише останній прихований стан h_n :

$$\frac{\partial y}{\partial h_n} = W_{hy}$$

Тепер у нас є все необхідне, щоб реалізувати зворотне розповсюдження в часі ВРТТ та закінчити `backprop()`:

```
1 class RNN:
2     # ...
3
4     def backprop(self, d_y, learn_rate=2e-2):
5         """
6         Выполнение фазы обратного распространения RNN.
7         - d_y (dL/dy) имеет форму (output_size, 1).
8         - learn_rate является вещественным числом float.
9         """
10        n = len(self.last_inputs)
11
12        # Вычисление dL/dWhy и dL/dby.
13        d_Why = d_y @ self.last_hs[n].T
14        d_by = d_y
15
16        # Инициализация dL/dWhh, dL/dWxh, и dL/dbh к нулю.
17        d_Whh = np.zeros(self.Whh.shape)
18        d_Wxh = np.zeros(self.Wxh.shape)
19        d_bh = np.zeros(self.bh.shape)
20
21        # Вычисление dL/dh для последнего h.
22        d_h = self.Why.T @ d_y
23
24        # Обратное распространение во времени.
25        for t in reversed(range(n)):
26            # Среднее значение: dL/dh * (1 - h^2)
27            temp = ((1 - self.last_hs[t + 1] ** 2) * d_h)
28
29            # dL/db = dL/dh * (1 - h^2)
30            d_bh += temp
31
32            # dL/dWhh = dL/dh * (1 - h^2) * h_{t-1}
33            d_Whh += temp @ self.last_hs[t].T
```

```

34
35     #  $dL/dW_{xh} = dL/dh * (1 - h^2) * x$ 
36     d_Wxh += temp @ self.last_inputs[t].T
37
38     # Далее  $dL/dh = dL/dh * (1 - h^2) * Whh$ 
39     d_h = self.Whh @ temp
40
41     # Отсекаем, чтобы предотвратить разрыв градиентов.
42     for d in [d_Wxh, d_Whh, d_Why, d_bh, d_by]:
43         np.clip(d, -1, 1, out=d)
44
45     # Обновляем вес и смещение с использованием градиентного спуска.
46     self.Whh -= learn_rate * d_Whh
47     self.Wxh -= learn_rate * d_Wxh
48     self.Why -= learn_rate * d_Why
49     self.bh -= learn_rate * d_bh
50     self.by -= learn_rate * d_by

```

Зауваження. У цьому коді є помилка. У частині `self.Whh` слід транспонувати матрицю `Whh`, тобто, код має виглядати так (виділено червоним кольором):

```

# Далі  $dL/dh = dL/dh * (1 - h^2) * Whh$ 
d_h = self.Whh.T @ temp

```

Моменти, на які необхідно звернути увагу:

- Ми об'єднали $\frac{\partial L}{\partial y} * \frac{\partial y}{\partial h}$ в $\frac{\partial L}{\partial h}$ для зручності;
- Ми постійно оновлюємо змінну `d_h`, яка містить останню версію $\frac{\partial y}{\partial h_{t+1}}$, що вимагає для підрахунку $\frac{\partial L}{\partial h_t}$;
- Закінчивши зі зворотним розповсюдженням у часі, ми використовуємо функцію [`np.clip\(\)`](#) на значеннях градієнту нижче -1 або вище 1. Це допоможе позбутися проблеми **вибухових градієнтів**. Таке трапляється, коли градієнти стають занадто великими через величезну кількість помножених параметрів. Вибух, а також зникнення градієнтів (vanishing gradient problem) не є рідкістю для класичних рекурентних нейронних мереж. Більш складні рекурентні нейронні мережі, наприклад LSTM, краще підійдуть для їх обробки;
- Коли всі градієнти підраховані, ми оновлюємо параметри ваги та зміщення за допомогою градієнтного спуску.

Тепер наша рекурентна мережа готова.

9. Тестування RNN

Протестуємо готову рекурентну нейронну мережу. Для початку, напишемо допоміжну функцію для обробки даних рекурентної нейронної мережі, що розглядається:

```
1 import random
2
3
4 def processData(data, backprop=True):
5     """
6     Возврат потери рекуррентной нейронной сети и точности для данных
7     - данные представлены как словарь, что отображает текст как True или False
8     - backprop определяет, нужно ли использовать обратное распределение
9     """
10    items = list(data.items())
11    random.shuffle(items)
12
13    loss = 0
14    num_correct = 0
15
16    for x, y in items:
17        inputs = createInputs(x)
18        target = int(y)
19
20        # Прямое распределение
21        out, _ = rnn.forward(inputs)
22        probs = softmax(out)
23
24        # Вычисление потери / точности
25        loss -= np.log(probs[target])
26        num_correct += int(np.argmax(probs) == target)
27
28        if backprop:
29            # Создание dL/dy
30            d_L_d_y = probs
31            d_L_d_y[target] -= 1
32
33            # Обратное распределение
34            rnn.backprop(d_L_d_y)
35
36    return loss / len(data), num_correct / len(data)
```

Тепер напишемо цикл для тренування мережі:

```

1 # Цикл тренування
2 for epoch in range(1000):
3     train_loss, train_acc = processData(train_data)
4
5     if epoch % 100 == 99:
6         print('--- Epoch %d' % (epoch + 1))
7         print('Train:\tLoss %.3f | Accuracy: %.3f' % (train_loss, train_acc))
8
9         test_loss, test_acc = processData(test_data, backprop=False)
10        print('Test:\tLoss %.3f | Accuracy: %.3f' % (test_loss, test_acc))

```

Результат виводу main.py виглядає наступним чином:

```

1 --- Epoch 100
2 Train: Loss 0.688 | Accuracy: 0.517
3 Test: Loss 0.700 | Accuracy: 0.500
4 --- Epoch 200
5 Train: Loss 0.680 | Accuracy: 0.552
6 Test: Loss 0.717 | Accuracy: 0.450
7 --- Epoch 300
8 Train: Loss 0.593 | Accuracy: 0.655
9 Test: Loss 0.657 | Accuracy: 0.650
10 --- Epoch 400
11 Train: Loss 0.401 | Accuracy: 0.810
12 Test: Loss 0.689 | Accuracy: 0.650
13 --- Epoch 500
14 Train: Loss 0.312 | Accuracy: 0.862
15 Test: Loss 0.693 | Accuracy: 0.550
16 --- Epoch 600
17 Train: Loss 0.148 | Accuracy: 0.914
18 Test: Loss 0.404 | Accuracy: 0.800
19 --- Epoch 700
20 Train: Loss 0.008 | Accuracy: 1.000
21 Test: Loss 0.016 | Accuracy: 1.000
22 --- Epoch 800
23 Train: Loss 0.004 | Accuracy: 1.000
24 Test: Loss 0.007 | Accuracy: 1.000
25 --- Epoch 900
26 Train: Loss 0.002 | Accuracy: 1.000
27 Test: Loss 0.004 | Accuracy: 1.000
28 --- Epoch 1000
29 Train: Loss 0.002 | Accuracy: 1.000
30 Test: Loss 0.003 | Accuracy: 1.000

```

Повний текст остаточної редакції програм доступний за посиланням [GitHub](#).

10. Завдання.

1. Вивчити побудову та навчання RNN на розглянутому прикладі;

2. Побудувати та навчити свою RNN для вирішення завдання, на вибір:

- класифікація коротких спадних, зростаючих чи періодичних послідовностей чисел;
- класифікація фраз технічного чи ліричного тексту;
- класифікація страв чи коктейлів за короткими рецептами;

або придумайте свій приклад.

3. Зробіть висновок: які гіперпараметри та їх значення ви використовували для побудови та навчання мережі, з якими труднощами вам довелося зіткнутися.

11. Зміст звіту:

- Формулювання завдання класифікації;
- Створені набори даних;
- Вибрана архітектура мережі, функції активації, гіперпараметри (обґрунтуйте);
- Коди програм із коментарями;
- Скріншоти результатів класифікації;
- Файли працюючих програм;
- Висновки.